

# **SC3099**

# **Capstone Project**

**Tianwei Zhang**

**College of Computing and Data Science**  
**Nanyang Technological University**

# Teaching Team

---

## Course Coordinator:

- ▶ Zhang Tianwei (Associate Professor @ CCDS): [tianwei.zhang@ntu.edu.sg](mailto:tianwei.zhang@ntu.edu.sg)

## Project Designer:

- ▶ Dr. Deng Gelei (Research Assistant Professor @ NTU):  
[gelei.deng@ntu.edu.sg](mailto:gelei.deng@ntu.edu.sg)

## Teaching Assistants (PhD students @ NTU):

- ▶ GOH XIN DEIK (Group 1-10): [GOHX0043@e.ntu.edu.sg](mailto:GOHX0043@e.ntu.edu.sg)
- ▶ KORKMAZ MEHMET ANIL (Group 11-20): [MEHMETAN001@e.ntu.edu.sg](mailto:MEHMETAN001@e.ntu.edu.sg)
- ▶ LIU SHUZHI (Group 21-30): [SHUZHI001@e.ntu.edu.sg](mailto:SHUZHI001@e.ntu.edu.sg)
- ▶ SZE CHUN KIT (Group 31-40): [CHUNKIT001@e.ntu.edu.sg](mailto:CHUNKIT001@e.ntu.edu.sg)

# Project Vision

---

Build a next-generation attendance verification system

Real-world challenges you will tackle:

- ▶ Deepfake detection and face spoofing prevention
- ▶ Replay attack mitigation
- ▶ GPS spoofing detection
- ▶ Remote/proxy sign-in prevention

This is what companies like Google, Apple, and banks deal with daily

# What You Will Build

---

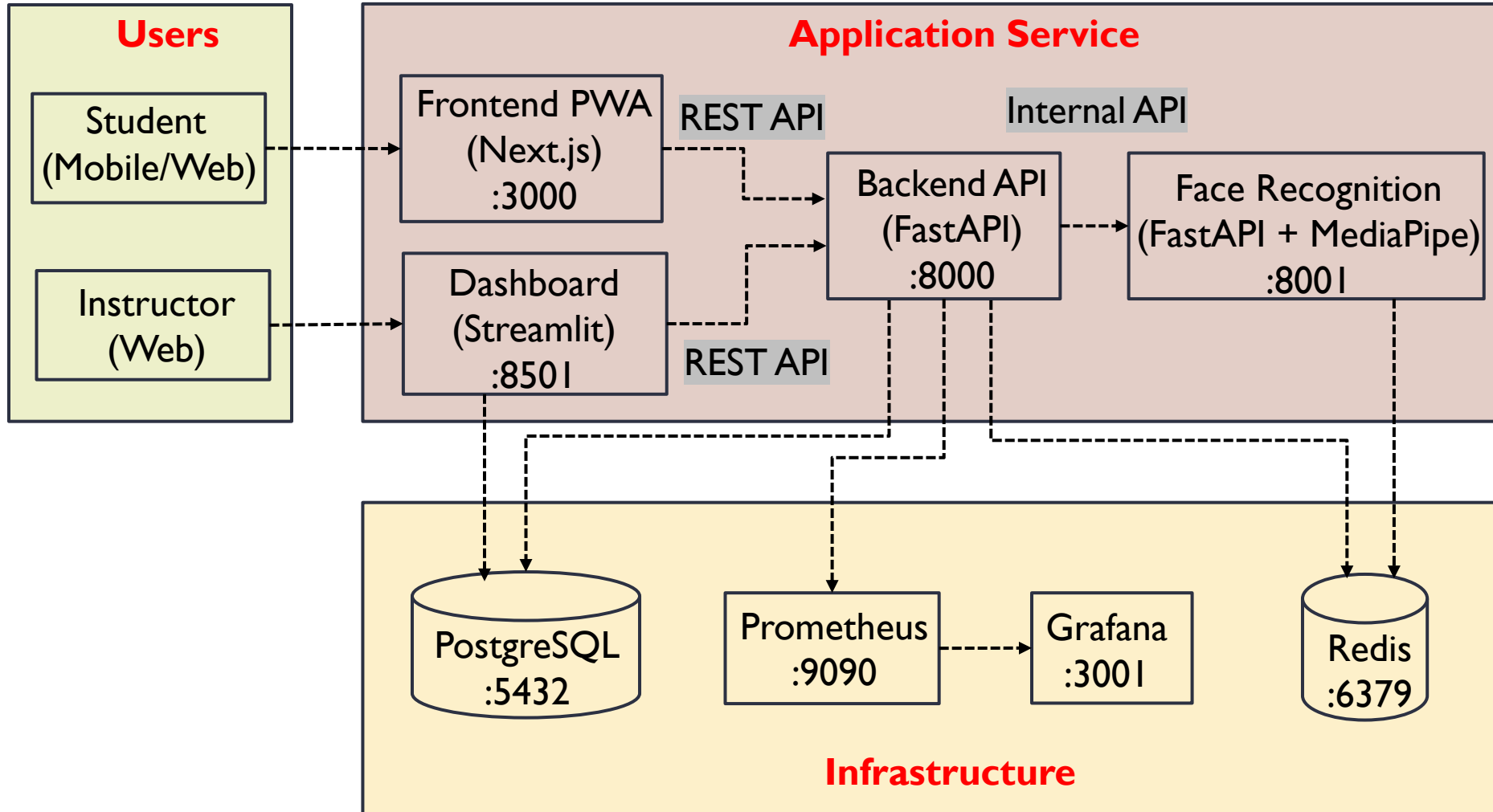
## A complete, production-grade attendance system

- ▶ Student mobile app with face recognition
- ▶ Backend API with security controls
- ▶ Machine learning service for face verification
- ▶ Instructor dashboard for monitoring

## End-to-end system from frontend to database

- ▶ 4 modules working together as microservices
- ▶ Deployable with Docker containers

# System Architecture



# What You Will Learn

---

## Full-stack web development

- ▶ REST API design and implementation
- ▶ Frontend/backend integration

## Security engineering

- ▶ JWT authentication, bcrypt hashing
- ▶ Rate limiting, input validation
- ▶ GDPR-like compliance

## Machine learning integration

- ▶ Face recognition

## DevOps & Observability

- ▶ Runtime logging and monitoring

# Teamwork & Contact Hours

---

## Team size: 4 students

- ▶ Recommended: 1 person responsible for one module
- ▶ All team members should understand the full system
- ▶ Work with your teammates to determine a good distribution.
- ▶ It is OK to rebalance/redistribute work as you go along

## Deliverable: working docker-deployable system

## Weekly lab sessions: 9:30-11:20 every Friday, LT26

- ▶ Week 1: briefing & Meet your team
- ▶ Week 2-12: group meeting and discussion.

# Attendance Requirements

---

Overall attendance [week 2-12]  $\geq 80\%$  required to pass

- ▶ We will use Wooclap to record your attendance for each lab session.
- ▶ In a random moment within the lab (9:30-11:20am), a barcode will be shown on screen. Use your phone to scan this barcode, and login with your NTU account.
- ▶ Responses must be submitted within 5 minutes of barcode being displayed.
- ▶ Exception will be given for students who have some technical problems with the login. Then please come to the stage and register manually
- ▶ Those who DO NOT fulfill the criteria will not be graded regardless of your team's performance, and will be required to retake capstone project.
- ▶ You may only know about it towards the end-of-semester.



# Assessment and Timeline

---

| Group assessment (60%)                            |     |                              |
|---------------------------------------------------|-----|------------------------------|
| System functionality testing<br>(90 public tests) | 5%  | Week 12                      |
| System feature testing<br>(40 hidden tests)       | 30% | Week 12                      |
| Live presentation                                 | 25% | Week 12                      |
| Individual assessment (40%)                       |     |                              |
| Early-stage peer review                           | 5%  | Week 7                       |
| Final-stage peer review                           | 15% | Week 12                      |
| Individual quiz                                   | 20% | 9:30-10:20am, Friday, Week 7 |

# System Functionality and Feature Testing

---

## System functionality testing (5%)

- ▶ Important functionality that you will need to complete your system, including mobile app, security controls, machine learning algorithms, monitoring mechanisms, etc.
- ▶ 90 public tests in the codebase. You can run these during development, to validate your implementation.
- ▶ For assessment, we will run your code and see how many tests it can pass.

## System feature testing (30%)

- ▶ Test if your implementation can achieve different features.
- ▶ 40 hidden tests: not revealed to you. We use them to test edge cases and advanced features.

## HTTP-based testing:

- ▶ Tests send requests to your running services
- ▶ Implementation-agnostic: any language works.

# Live Presentation (25%)

---

## Report achievements and contributions

- ▶ In lieu of a traditional written report + presentation
- ▶ Demonstrate novelty, creativity, presentation skills, teamwork, etc.
- ▶ 10 minutes presentation + 3 minutes QA
- ▶ Will be scheduled in Week 12

## Content to be featured

- ▶ Team members: introduce team members and responsibilities assigned.
- ▶ Implementations: highlight and communicate the team's implementation of various aspects of the project, covering different modules as well as their integration as an entire system
- ▶ Special achievements: highlight what else is special about your team's effort and the solutions that you have implemented.

# Peer Evaluation (5% + 15%)

Assess contributions to the design, planning and implementation phases. Main aim is to prevent free-loaders in the team.

- ▶ Confidential and anonymous
- ▶ Two stages: Week 7 (5%) and Week 12 (15%)
- ▶ Students who get a peer evaluation score of less than 50% compared to the rest of the members can have their final grade greatly reduced as compared to the rest of the teammates.
- ▶ Important for each student to keep a record of his contribution to the team to assist our investigation in the event of issues raised from the teammates.

## Key points

- ▶ You need to provide comments or valid reasons if the score is less than 5/10.
- ▶ Comments are also important when the score is 10/10.
- ▶ Scores without valid comments cannot be taken into consideration and moderation will be done if necessary comments are unavailable.

# Individual Quiz (20%)

Quiz is divided into the following four topics that represent the major modules of the project.

- ▶ Mobile app with face recognition
- ▶ Backend API with security controls
- ▶ Machine learning service for face verification
- ▶ Instructor dashboard for monitoring.

## Quiz format and arrangement

- ▶ All MCQ.
- ▶ Each student must choose a single quiz topic to take, and each group must ensure that all the 4 topics are taken by the members.
- ▶ Quiz content will be based on the tasks and functionalities related to the system. You are expected to demonstrate good understanding of the tools you have used to implement the required functionalities and features.
- ▶ Duration is 30 minutes. It is closed-book, and will be taken using the NTULearn online assessment.

**More details about the system**

# System Architecture

---

4 Modules. Each module is a separate service that communicates over HTTP.

## Module 1: Frontend PWA (Student App).

- ▶ Camera access, geolocation, check-in interface.

## Module 2: Backend API (Core Business Logic)

- ▶ Authentication, courses, sessions, check-ins.

## Module 3: Face Recognition Service (ML)

- ▶ Face enrollment, verification, liveness detection.

## Module 4: Observability Dashboard (Instructor Portal)

- ▶ Session management, analytics, data export.

# Technology Freedom

---

## You can choose any tools.

- ▶ Backend: FastAPI, Express.js, Spring Boot, Laravel, Go, etc.
- ▶ Frontend: React, Vue, Angular, Svelte, vanilla JS, etc.
- ▶ Database: PostgreSQL (recommended), MySQL, etc.
- ▶ Face ML: MediaPipe, face-recognition, TensorFlow, etc.
- ▶ Dashboard: Streamlit, Dash, React, Django, etc.
- ▶ ANY programming language that can serve HTTP endpoints!
- ▶ Python, Java, JavaScript, PHP, Go, Rust, C# - all welcome.

## Why Technology-agnostic?

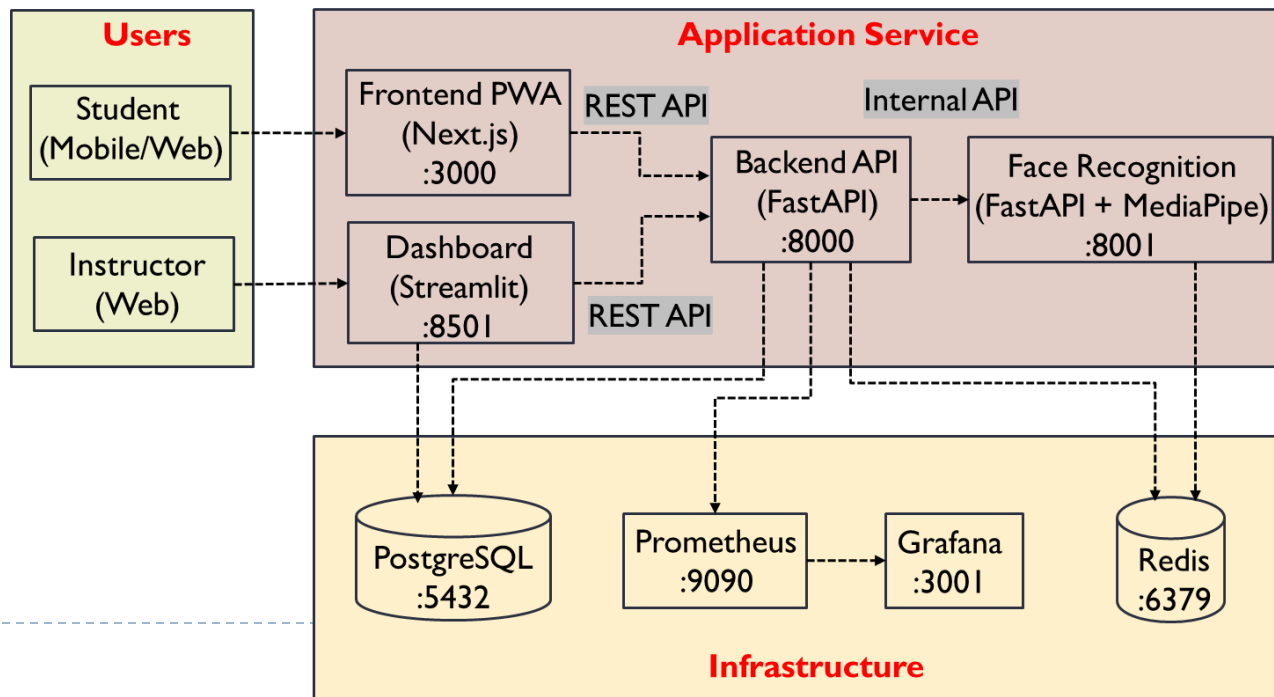
- ▶ Simulates real-world software development: Companies don't dictate your exact tech stack; Focus on solving problems, not specific frameworks
- ▶ Tests only check HTTP endpoint behavior: Black-box testing approach; Your internal code is YOUR business
- ▶ Encourages architectural thinking: How do services communicate? What contracts must be honored?



# Service Communication Flow

Understand this flow thoroughly.

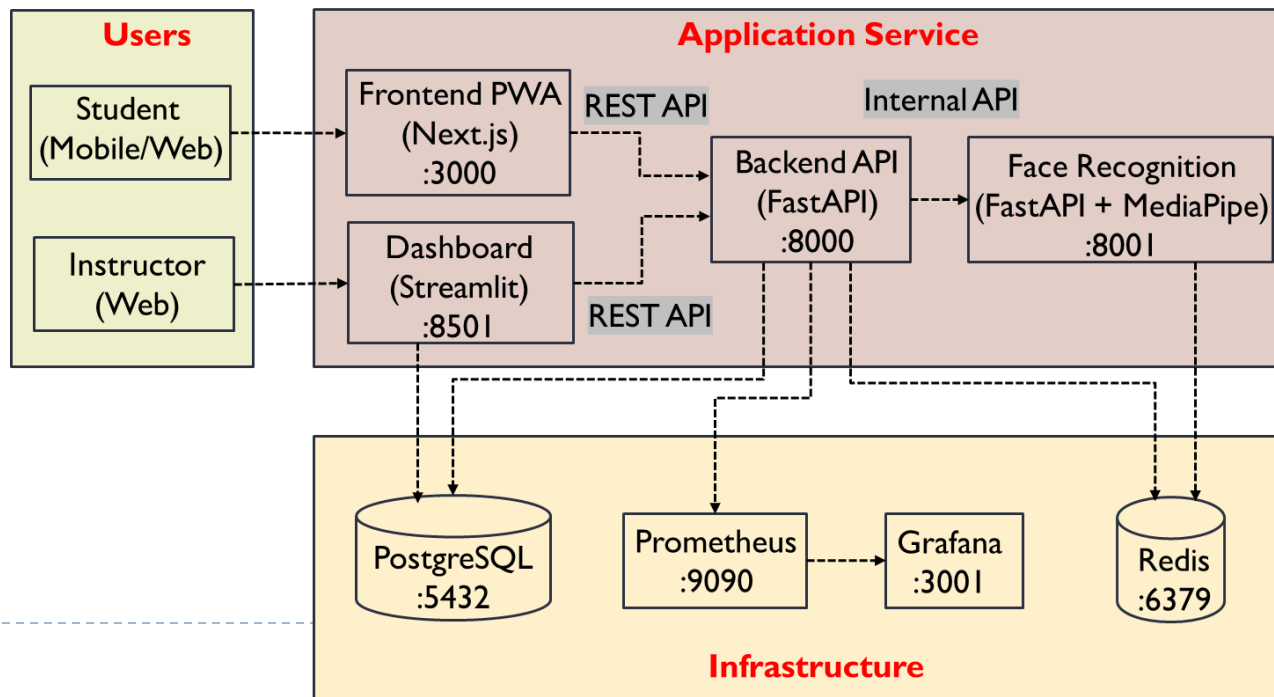
- ▶ All requests go through the backend.
- ▶ It orchestrates calls to the face service and database.
- ▶ This separation of concerns is a key architectural pattern



# Infrastructure Provided (docker-compose)

## Key components

- ▶ PostgreSQL database (port 5434): persistent storage for all data
- ▶ Redis (port 6380): caching and rate limiting
- ▶ Prometheus (port 9090): metrics collection
- ▶ Grafana (port 3001): metrics visualization (admin/admin)
- ▶ OpenTelemetry Collector: distributed tracking support



# Module 1: Frontend PWA

---

## Progressive Web App for students

### Key features to implement

- ▶ User authentication (login/register forms)
- ▶ Camera access with WebRTC (browser API for camera access)
- ▶ Liveness challenge UI (blink, head turn), preventing photo attacks.
- ▶ Geolocation consent and capture
- ▶ Device fingerprinting
- ▶ Check-in workflow (session -> camera -> location -> submit)

### Must work on mobile browsers

# Module 2: Backend API

---

Core business logic - the heart of the system

## Key responsibilities

- ▶ JWT authentication & token refresh
- ▶ Role-based access control (student/ta/instructor/admin)
- ▶ User, Course, Session, Enrollment management
- ▶ Check-in processing with risk assessment
- ▶ Rate limiting (Redis-based)
- ▶ Audit logging for compliance

~50 API endpoints to implement

# Module 3: Face Recognition Service

---

## Machine learning microservice

- ▶ Use MediaPipe or similar libraries for face detection and landmark extraction

## Key endpoints

- ▶ POST /face/enroll - Generate face template
- ▶ POST /face/verify - Match face to template
- ▶ POST /liveness/check - Detect spoofing

## CRITICAL privacy requirement

- ▶ NO raw face images stored anywhere
- ▶ Only hashes of face templates
- ▶ Images processed in memory only

# Module 4: Observability Dashboard

---

Instructor analytics portal: observe who checks in, identify suspicious activities, and export attendance for grading

## Key features

- ▶ Login with instructor credentials
- ▶ Session management (create, activate, close)
- ▶ View all check-ins for a session
- ▶ Review flagged check-ins (approve/reject)
- ▶ Attendance statistics and charts
- ▶ Export data to CSV/JSON for gradebook

Real-time updates preferred

# Database Schema

---

## 8 Tables

- ▶ users - Student/instructor accounts with roles
- ▶ courses - Course metadata, geofence settings
- ▶ enrollments - Student-course relationships
- ▶ sessions - Class sessions with check-in windows
- ▶ checkins - Attendance records with risk scores
- ▶ devices - Device binding for security
- ▶ risk\_signals - Multi-signal risk data per check-in
- ▶ audit\_logs - Immutable event logs for compliance: once written, they cannot be modified or deleted.

Full schema in [docs/DATABASE-SCHEMA.md](#)

# Security: Authentication & Authorization

---

## JWT-based authentication

- ▶ Access token: 1 hour expiry
- ▶ Tokens should never contain sensitive data. They are only for authentication
- ▶ Refresh token: 7 days expiry
- ▶ HS256 signing algorithm
- ▶ Ignore security restrictions about the roles during registration.

## Password requirements:

- ▶ Never store plaintext passwords!
- ▶ Minimum 8 characters
- ▶ bcrypt hashing with cost  $\geq 10$ : the hash takes about 100ms to compute, which slows down brute force attacks.

## Role-based access control (RBAC):

- ▶ student, ta, instructor, admin
- ▶ Each endpoint has role requirements
- ▶ Check API-SPECIFICATION.md for detailed access rules of each role.



# Security: Rate Limiting & Validation

---

## Rate limiting: prevent brute-force attacks and DoS

- ▶ Use Redis to track requests counts with TTL-based expiry.
- ▶ Login: 60 attempts/hour per IP
- ▶ API requests: 1000/hour per user
- ▶ Check-ins: 10/minute per user
- ▶ Registration: 10/hour per IP

## Input validation:

- ▶ Prevent SQL injection
- ▶ Sanitize XSS payloads
- ▶ Validate email formats.

## Geofencing with Haversine distance formula

- ▶ Calculate distance between GPS coordinates on Earth's surface

# Security: Device Binding & Anti-Fraud

---

## Device binding

- ▶ Each student registers their device
- ▶ Check-ins from unknown devices flagged

## Liveness detection (anti-spoofing):

- ▶ Detect printed photos
- ▶ Detect screen replay attacks
- ▶ 3D depth analysis with face mesh.

## Risk scoring algorithm

- ▶ Combine multiple signals, including liveness, location, device trust, timing signals
- ▶ Flag high-risk check-ins for review

## Impossible travel detection

---



# Privacy: Design Principles

---

## GDPR-like compliance built into the design

### Key principles

- ▶ Data minimization - collect only what's needed
- ▶ Purpose limitation - use data only for stated purpose
- ▶ Explicit consent - users must opt-in
- ▶ Right to deletion - users can request data removal

### Consent tracking

- ▶ Camera consent flag
- ▶ Geolocation consent flag
- ▶ Both must be TRUE before check-in

# Privacy: Data Handling Rules

---

## Face data

- ▶ NO raw face images stored
- ▶ Only hashes (64 hex characters)
- ▶ Images processed in memory, then deleted

## Location data

- ▶ Only used for geofence check
- ▶ Stored with limited precision

## Personal data

- ▶ 30-day auto-deletion policy
- ▶ `scheduled_deletion_at` field in users table.

# Privacy: Audit & Compliance

---

## Immutable audit logs

- ▶ Cannot be modified or deleted
- ▶ Using append-only tables or write-once storage.
- ▶ Track all security-relevant events

## 18 tracked action types

- ▶ login\_success, login\_failed, logout
- ▶ checkin\_approved, checkin\_flagged, checkin\_rejected
- ▶ user\_created, face\_enrolled, device\_registered
- ▶ And more...

## Export capability for compliance audits

# System Functionality: 90 Public Tests

---

| Test Category      | Points | Focus Area                                           |
|--------------------|--------|------------------------------------------------------|
| API Functional     | 26     | CRUD operations, authentication, check-ins           |
| Face Recognition   | 15     | Enrollment, matching, liveness, privacy              |
| Security Basic     | 12     | Auth, authorization, input validation, rate limiting |
| Observability      | 12     | Dashboard stats, session management, export          |
| Privacy Basic      | 8      | Consent, data minimization, retention                |
| Frontend/Dashboard | 8      | Contract compliance, integration                     |
| Performance        | 5      | Latency < 2s, 10 concurrent users                    |
| Integration        | 4      | End-to-end workflows                                 |

# System Feature: 40 Hidden Tests

---

| Test Category                | Points | Focus Area                                                         |
|------------------------------|--------|--------------------------------------------------------------------|
| Advanced Security            | 12     | GPS spoofing detection, Replay attack prevention, Network security |
| Privacy Audit                | 8      | Database inspection for raw images, Encryption verification        |
| Face Recognition<br>Advanced | 10     | Anti-spoofing with real attack images                              |
| Liveness                     | 3      |                                                                    |
| Stress Testing               | 7      | 100 concurrent users                                               |

# How Testing Works

---

## Tests are black-box HTTP requests

- ▶ No code inspection - only endpoint behavior matters
- ▶ Tests create their own data via API calls
- ▶ Admin endpoints required for test setup.

## Environment setup

- ▶ `export TEST_BACKEND_URL=http://localhost:8000`
- ▶ `export TEST_FACE_URL=http://localhost:8001`

## Run tests

- ▶ `python3 -m pytest tests/public/ -v`



# Suggested Timeline for Development

---

| Phase               | Weeks | Focus                                |
|---------------------|-------|--------------------------------------|
| Foundation          | 1-2   | Environment Setup                    |
| Core Backend        | 2-4   | Basic Backend API and Authentication |
| ML Service          | 5-6   | Face Recognition Integration         |
| Backend Integration | 7     | Integrate API with Face Recognition  |
| User Interfaces     | 8-9   | Frontend PWA & Dashboard             |
| Polish              | 10    | Performance & Privacy                |
| Hardening           | 11    | Edge Cases & Hidden Tests            |
| Presentation        | 12    | Live Presentation & Documentation    |

# Suggested Timeline for Development

---

## Week 1: Project setup

- ▶ Set up development environment (Docker, databases)
- ▶ Implement basic user registration and login
- ▶ **Target: Get familiar with the target system**

## Week 2: Authentication, user management & authorization

- ▶ Implement basic login feature
- ▶ Role-based access control (student, TA, instructor, admin)
- ▶ User profile management
- ▶ JWT-based authentication
- ▶ **Target: RBAC system complete.**

# Suggested Timeline for Development

---

## Week 3: Courses & Sessions

- ▶ Course CRUD operations
- ▶ Session management with status workflow
- ▶ Enrollment system
- ▶ **Target: Course and session APIs functional**

## Week 4: Check-in Core

- ▶ Check-in submission endpoint
- ▶ Geofence validation (distance calculation)
- ▶ Check-in status logic (approved/flagged/rejected)
- ▶ **Target: Basic check-in flow working**

# Suggested Timeline for Development

---

## Week 5: Face recognition - Core

- ▶ Set up face recognition service
- ▶ Face detection with MediaPipe
- ▶ Face enrollment and representation generation
- ▶ **Target: Face enrollment working**

## Week 6: Face recognition - Advanced

- ▶ Face verification/matching
- ▶ Risk assessment endpoint
- ▶ Backend integration with face service
- ▶ **Target: Full face recognition pipeline**

# Suggested Timeline for Development

---

## Week 7: Integration

- ▶ Connect all components in the backend API
- ▶ Connect database with the real face recognition system
- ▶ Integrate face recognition system into the API
- ▶ **Target: Student enroll faces and check in via backend**

## Week 8: Frontend PWA

- ▶ Authentication UI
- ▶ Camera capture (WebRTC)
- ▶ Geolocation integration
- ▶ Check-in flow
- ▶ **Target: Student can check in via mobile/web**

# Suggested Timeline for Development

---

## Week 9: Instructor dashboard

- ▶ Session management interface
- ▶ Attendance analytics
- ▶ Flagged check-in review
- ▶ Export functionality
- ▶ **Target: Instructors can manage attendance**

## Week 10: Performance & Privacy

- ▶ Database optimization (indexes, queries)
- ▶ Concurrent request handling
- ▶ Data retention policies
- ▶ Privacy compliance
- ▶ **Target: Pass performance and privacy tests**

# Suggested Timeline for Development

---

## Week 11: Hidden Tests & Edge Cases

- ▶ GPS spoofing detection
- ▶ Replay attack prevention
- ▶ Anti-spoofing (photos, screens)
- ▶ Stress testing (50+ concurrent users)
- ▶ Complete audit trail
- ▶ **Target: Maximize hidden test scores**

## Week 12: Presentation & Demo

- ▶ Prepare presentation slides
- ▶ Final documentation and code
- ▶ **Target: Conduct live presentation**

# Course Materials on NTULearn

---

## Group allocation

- ▶ Randomly assigned, following college policy.
- ▶ Does not support group reassignment

## Slides

- ▶ Briefing
- ▶ Technical materials for the designs of 4 modules

## Codebase

More will be added with the progress of this course



# Getting Started: Study the Codebase

---

## Documentation (docs/ folder)

- ▶ API-SPECIFICATION.md - Complete API documentation
- ▶ DATABASE-SCHEMA.md - All 8 tables defined
- ▶ SECURITY-REQUIREMENTS.md - Auth, rate limiting specs
- ▶ INTEGRATION-GUIDE.md - Module communication

## Sample images (sample\_images/)

- ▶ Face images for testing enrollment/matching

## Public test suite (tests/public/)

- ▶ 90 points of tests you can run

## Docker Compose for infrastructure

# Getting Started: Quick Start

---

1. Download the code repository
2. Read ALL documentation in docs/ folder
3. Start infrastructure:
  - ▶ `docker-compose up -d postgres redis`
4. Choose your tech stack for each module
5. Implement endpoints one category at a time
6. Run public tests frequently to validate
7. Integrate modules and test end-to-end
8. Iterate until all public tests pass

# Getting Started: Running Tests

---

## Install test dependencies:

- ▶ `pip install -r requirements-test.txt`

## Start your services

- ▶ `docker-compose up -d`

## Set environment variables

- ▶ `export TEST_BACKEND_URL=http://localhost:8000`
- ▶ `export TEST_FACE_URL=http://localhost:8001`

## Run public tests

- ▶ `python3 -m pytest tests/public/ -v`

# Tips: Common Pitfalls to Avoid

---

PRIVACY VIOLATION: Storing raw face images

SECURITY FAIL: Plaintext passwords

TEST FAILURE: Missing admin endpoints

TEST FAILURE: Forgetting session activation endpoint

- ▶ Sessions must be 'active' before check-ins work!

PERFORMANCE: N+1 database queries

## BEST PRACTICES:

- ▶ Read ALL documentation first
- ▶ Run tests early and often
- ▶ Implement admin endpoints first

# What Do You Do Today?

---

Find your teammates.

Start discussing division of responsibilities.

Determine who gets what part of the system to play with, etc.

Let us know if your teammates cannot be approached.

Timing from week 2 onwards: Friday 9:30 – 11:20am, LT26.

# Seat Map

---

Group I - 10

TA: GOH XIN DEIK

Group II - 20

TA: KORKMAZ  
MEHMET ANIL

Group 2I - 30

TA: LIU SHUZH

Group 3I - 40

TA: SZE CHUN KIT

Platform

**Any Questions?**